

Practical Exam Topics

- Values and Types
- Operations
- Strings
- ASCII
- Input
- Printing / Returning
- Loops
- Conditionals
- Lists
- Dictionaries
- 1 hr, 4 tasks

Write a function that adds up multiples of 9

- the function should
 - ask the user for input
 - let the user know how to end the program
 - check whether the input is a number that is a multiple of 9
 - add multiples of 9 to a running total
 - return the total
- start with the most simple working version, then add complexities
- this activity should take around 15 mins

```
1 # Georgia Doing
2 # Practice Problem 1 for Practical Exam 2
3
4 def sum_nines():
5     # takes user input, adding up multiples of 9
6     # takes no parameters
7
8     # initialize accumulator
9     usr_sum = 0
10    # set indicator variable to start loop
11    go = True
12    while go:
13        # ask for a number or 'done'
14        usr_in = input("Give me numbers and I'll count 9s. Or type 'done' to exit.")
15        # check if loop should exit, robust to 'Done'
```

```

16     if usr_in.lower() == "done":
17         go = False
18         # use mod operator to check if div by 9
19         elif int(usr_in) % 9 == 0:
20             usr_sum += int(usr_in)
21         else:
22             print("num not div by 9")
23     # return rather than print answer
24     return usr_sum
25
26 sum_nines()

```

```

1 def num_nines():
2     total=0
3     x=1
4     while x>=0:
5         x=int(input("Input a number. input -1 to end program"))
6         if x%9==0:
7             #if you can divide by 9 with no remainder, add it to total
8             total+=x
9     return total
10
11 print(num_nines())

```

```

1 def sum_nines():
2     #defining the functionw ith no parameters
3
4     keep_playing = "y"
5     #initializing a variable that will hold if the user wants to keep enterig
6     #numbers
7
8     total = 0
9     #initializing the accumulator variable that will sum the multiples of nines
10
11     while keep_playing[0] == "y":
12         #as long as the user wants to play:
13
14         user_input = int(input("Please enter a number: "))
15         #ask the user the number they want to enter and save in a variable
16
17         if user_input % 9 == 0:
18             #if the user enters a multipe of 9:
19
20             total += user_input
21             #add that number tot the total
22         keep_playing = input("Would you like to enter a number? Type y or n: ")
23         #asking if the user wants to play again

```

```
24
25     print("Thanks for playing!")
26     #When the user is done playing display this exit message
27
28     return total
29     #returning the total
30
31 ###BEGIN TESTING
32
33 print(sum_nines())
```

```
1 def sum_nines():
2     #Function will ask the user for input and how to end the program
3     #Check whether the input is a number that is a multiple of 9
4     #add multiples of 9 to running total
5     #return total
6
7     total = 0
8     user_input= ""
9
10    print("Enter a number to check whether it is a multiple of 9 or not.")
11    print("Type 'q' to quit the program and see the total")
12
13    while user_input != "q":
14        user_input= input("Enter a number to check whether it is a multiple of 9 or
↵ not")
15        number = int(user_input)
16        if number % 9 ==0:
17            total = total+number
18            print(number, "is a multiple of 9! Added to total.")
19        else:
20            print(number, "is NOT a multiple of 9.")
21    else:
22        print("Please enter a number or 'q' to quit.")
23
24    return total
25
26
27 ###BEGIN TESTING
28
29
30
31 result = sum_nines()
```

Write a function that makes lists alliterative

- the function should:
 - take a lists of words and a character as parameters
 - return a new list only containing words that start with the character
 - be robust to edge cases (empty lists, maybe lists without strings)
 - optionally alter the function to take an ASCII value instead of a char
- start with the most simple working version, then add complexities
- this activity should take around 15 mins

```

1  # Georgia Doing
2  # Practice Problem 2 for Practical Exam 2
3
4  def al_list(alist, achar):
5      # filter lists of words to be alliterative
6      # param alist: a list of words/strings
7      # param achar: a character for selection
8      new_list = []
9      for word in alist:
10         if not type(word) == type("anystring"):
11             print("skipping item", word)
12         else:
13             if word[0] == achar:
14                 new_list.append(word)
15     return new_list
16
17 print(al_list(["ginny","georgia","dog","goose",5],"g"))

```

```

1  def al_list(item,word_list):
2      """ this fucntion takes a word list and item as peramiters
3      it then isolates each word in the list and checks to see if
4      the first letter is the same as the item. if it is it saves
5      it to new list"""
6      # here we definine the new empty list that will be saved to
7      new_list = []
8      # here we are gonna check for an empty list
9      if word_list == []:
10         print("that list was empty and wont work")
11         return none
12     #here is the for loop that isolates each word in the string
13     for word in word_list:
14         # we first make sure that anything in that string is an int
15         string_word = str(word)
16         # then we pick out the first letter of the word
17         first_letter = word[:1]

```

```

18     # we then check if that first letter is the same as we are given
19     if first_letter == item:
20         # if it is we append the word list
21         new_list.append(word)
22     print(new_list)
23
24
25 ### BEGIN TESTING
26
27 x = ["apple","tree","pinapple","already"]
28
29 al_list("a",x)

```

```

1 def al_lists(list_of_words,ch):
2     '''al_lists takes parameters list_of_words and ch, using them to make lists
3     ↪ alliterative'''
4
5     length=len(list_of_words)
6
7     al_list=[]
8
9     #accounts for empty word_list
10    if length==0:
11        return al_list
12
13    #this for loop checks each word in a word list to see if the ch is the first
14    ↪ letter
15    for line in range (0,length):
16
17        if ch in list_of_words[line][0]:
18
19            #adds this word to al_list
20            al_list.extend(list_of_words[line])
21
22    return al_list
23
24 ###BEGIN TESTING###
25
26 #example
27 word_list=[["brain"],["barber"],["crop"]]
28
29 print(al_lists(word_list,"b"))

```

```

1 def al_lists(thelist, char):
2     new_list = []
3     for word in thelist:
4         split_word = list(word)

```

```

5         if split_word[0] == char:
6             new_list.extend([word])
7
8
9
10        return new_list
11
12    ###BEGIN TESTING
13
14    mylist = ["hair", "corn", "helmet", "cake", "home"]
15
16    print(al_lists(mylist, "h"))
17
18    print(al_lists(mylist, "c"))

```

```

1  def al_lists(your_list,character):
2      new_list=[]
3      for word in your_list:
4          if word[0]== character:
5              new_list.append(word)
6              print(new_list)
7      return new_list
8
9  ##BEGIN TESTING
10
11  my_list=["cool","chill","cold","yes"]
12  al_lists(my_list,"c")

```

Change the function above to take an ascii value instead of a character

- the function should:
 - take a lists of words and an integer as parameters
 - return a new list only containing words that start with the ascii value character
- start with the most simple working version, then add complexities
- this activity should take around 5 mins

```

1  # Georgia Doing
2  # Practice Problem 3 for Practical Exam 2
3
4  def ascii_lists(alist, val):
5      # filter lists of words to be alliterative
6      # param alist: a list of words/strings
7      # param value: an ascii value for selection
8      # returns list

```

```

9
10 new_list = []
11 # check val is valid ascii
12 if val < 0 or val > 255:
13     print("val outside ascii range")
14     return new_list
15 # if val is valid, check against words
16 for word in alist:
17     if not type(word) == type("anystring"):
18         print("skipping item", word)
19     else:
20         # convert from ascii value to character
21         if word[0] == chr(val):
22             new_list.append(word)
23 return new_list
24
25 print(ascii_lists(["ginny","georgia","dog","goose",5],ord("g")))

```

```

1 def ascii_lists(word_list, ascii_value):
2     """take a word list (parameter 1) and an integer (parameter 2) and
3     return a new list only containing words starting with the ascii value
4     ↪ character"""
5     # if the word list is empty, return None
6     if word_list == []:
7         return None
8     # make a copy of the word list
9     words = word_list.copy()
10    # otherwise, create a new empty list
11    new_list = []
12    character = chr(ascii_value)
13    # traverse through each element in the word list
14    for element in words:
15        # if the element is a list, delete the element (it is not a string)
16        if element is list:
17            del element
18        # otherwise:
19        else:
20            # convert the element to a string (it could be int or bool)
21            element = str(element)
22            # if the first character in the element is the same as the character
23            ↪ (parameter 2)
24            if element[0] == character:
25                # append the element to the new list
26                new_list.append(element)
27
28    # if there are no words in the word list that start with the character, return
29    ↪ None

```

```

27     if new_list == []:
28         return None
29
30     # return the list of words that start with the ascii value character
31     return new_list
32
33
34 ### BEGIN TESTING
35
36 words = ['apple', 'potato', 'arugula', 'armenia', [1, 2, 3], ['apple', 'potato'],
37         ↪ True, False]
38 ascii_a = ord('a')
39 ascii_p = ord('p')
40 print(ascii_lists([], ascii_a))
41 print(ascii_lists(words, ascii_a))
42 print(ascii_lists(words, ascii_p))
43 print(ascii_lists(words, 60))

```

Write a program that helps the user study

- the program should:
 - initiate a dictionary of CSC106 topics and in-class activities
 - name activities based on their week
 - decide what info will be keys and what will be values
 - decide if you need to use strings, lists, nested lists, nested dictionaries, etc.
 - include a function that looks up activities based on a topic
 - optionally include information on whether you completed each activity
 - optionally include descriptions of each activity
- this activity should take around 20 mins
- W2-2 : for loops, input
- W2-3 : memory diagrams and tracing
- W3-3 : math facts
- W3-4 : conditionals
- W4-1 : multi-branch conditionals (Reeborg)
- W4-2 : conditionals
- W5-1 : while loops
- W5-2 : ASCII
- W6-1 : lists and strings
- W6-2 : Caesar cipher, strings

- W6-3 : lists, numbers and strings
- W7-1 : lists of numbers
- W7-2 : nested lists
- W7-3 : dictionaries and lists
- W8-1 : reading/writing files
- W8-2 : dictionaries for translation
- W8-3 : author identification

```
1 # Georgia Doing
2 # Practice Problem 4 for Practical Exam 2
3
4 study_guide = {
5     "W2-2" : "for loops",
6     "W2-3" : "memory diagrams" ,
7     "W3-3" : "math",
8     "W3-4" : "conditionals",
9     "W4-1" : "conditionals",
10    "W4-2" : "conditionals",
11    "W5-1" : "while loops",
12    "W5-2" : "ASCII",
13    "W6-1" : "strings",
14    "W6-2" : "strings",
15    "W6-3" : "strings",
16    "W7-1" : "lists",
17    "W7-2" : "lists",
18    "W7-3" : "dictionaries",
19    "W8-1" : "files",
20    "W8-2" : "dictionaries",
21    "W8-3" : "dictionaries"
22 }
23
24
25 def study_tool(topic):
26     if not topic in study_guide.values():
27         print("topic not covered, maybe take CSCS120!")
28         print("browse: ", study_guide.keys())
29         return
30
31     for k, v in study_guide.items():
32         if v == topic:
33             print(k)
34
35 study_tool("lists")
```

```

1 # Georgia Doing
2 # Practice Problem 2 with options for Practical Exam 2
3
4 study_guide = {
5     "W2-2" : {"topic": "for loops", "description" : "variables, arithmetics, printing",
6         ↪ "completed": True},
7     "W2-3" : {"topic": "memory diagrams", "description" : "tracing, debugging loops",
8         ↪ "completed": True},
9     "W3-3" : {"topic": "math", "description" : "math operators, check even",
10        ↪ "completed": True},
11     "W3-4" : {"topic": "conditionals", "description" : "if/elif/else, conversions,
12        ↪ calculators", "completed": True},
13     "W4-1" : {"topic": "conditionals", "description" : "multi-branch if statements,
14        ↪ logical operators", "completed": True},
15     "W4-2" : {"topic": "conditionals", "description" : "largest number, congress, leap
16        ↪ years, dice", "completed": True},
17     "W5-1" : {"topic": "while loops", "description" : "tracing, summing numbers",
18        ↪ "completed": True},
19     "W5-2" : {"topic": "ASCII", "description" : "ASCII to string and back, pig latin",
20        ↪ "completed": True},
21     "W6-1" : {"topic": "lists", "description" : "list tracing and operations, string
22        ↪ manipulation", "completed": True},
23     "W6-2" : {"topic": "strings", "description" : "Caesar cipher", "completed": True},
24     "W6-3" : {"topic": "strings", "description" : "lists of numbers and strings, cord
25        ↪ count, variance", "completed": True},
26     "W7-1" : {"topic": "lists", "description" : "list micro programs, minimum",
27        ↪ "completed": True},
28     "W7-2" : {"topic": "lists", "description" : "Nested lists (2D), row and column
29        ↪ sums", "completed": True},
30     "W7-3" : {"topic": "dictionaries", "description" : "phone book, votes, records,
31        ↪ word counts, accounts", "completed": True},
32     "W8-1" : {"topic": "files", "description" : "describe data, make randoms",
33        ↪ "completed": True},
34     "W8-2" : {"topic": "dictionaries", "description" : "english-german translation",
35        ↪ "completed": True},
36     "W8-3" : {"topic": "dictionaries", "description" : "author identification",
37        ↪ "completed": True}
38 }
39
40 def study_tool(topic):
41     all_topics = []
42     for d in study_guide.values():
43         all_topics.append(d["topic"])
44     if not topic in all_topics:
45         print("topic not covered, maybe take CSCS120!")
46         print("browse: ", study_guide.keys())

```

```

32     return
33
34     for k, v in study_guide.items():
35         if v["topic"] == topic:
36             print(k)
37             print(v["description"])
38             print("completed: ", v["completed"])
39
40 study_tool("lists")

```

```

1 def study_tool():
2     csc106= { 'for loops': 'W2-2',
3               'input': 'W2-2',
4               'memory diagrams': 'W2-3',
5               'tracing' : 'W2-3',
6               'math facts' : 'W3-3',
7               'conditionals' : ['W3-4', 'W4-2'],
8               'Reeborg' : 'W4-1',
9               'while loops': 'W5-1',
10              'ASCII' : 'W5-2',
11              'lists' : 'W6-1',
12              'strings' : ['W6-1', 'W6-2', 'W6-3'],
13              'Ceasar cipher' : 'W6-2'
14            }
15     topic=input("What topic do you want the activities for?")
16     topic=topic.lower()
17
18     if topic in csc106:
19         return csc106[topic]
20
21
22 print(study_tool())

```

```

1 # { topic:[day,complete,discription]}
2 def study_tool():
3
4     """ this is a study tool that asks a user for what they want to review then pulls
5         ↪ according to that input"""
6
7     #I cut this short but can go longer
8     topics = {"for loop":["W2-2.1","complete","for loops go through a set of codes for
9         ↪ an amount of time"],
10               "input" :["W2-2.2","complete","it prints something and gets a users
11         ↪ response"],
12               "memory diagrams" :["W2-3","complete","how we can trace out our work"]}
13
14     print("I am here to help you review, will print out a list of topics for possible
15         ↪ review")

```

```
12 print (" Do you have a topic that you want to study or did you miss a class?")
13 topic_or_class = input("if it a topic type T if it is a class type C")
14
15 if topic_or_class == "T":
16     print ("you want to study based upon topic. I will give you the topics to pick
17         ↪ from")
18     print("Please type it out exsactly as it is printed")
19     for keys in topics:
20         print(keys)
21     review = input("what would you like to review?")
22 elif topic_or_class == "C":
23     print("you want to review based on class please tell me the class you want to
24         ↪ review from your options below")
25     for keys in topics:
26         print(topics[keys][0])
27     week_day = input("what class did you miss")
28     for keys in topics:
29         if topics[keys][0] == week_day:
30             review = keys
31 else:
32     print("that was not one of the options")
33
34 for keys in topics:
35     if review == keys:
36         print("you wanted to review", keys)
37         print("that topic was done on", topics[keys][0])
38         print("you marked that the work was", topics[keys][1])
39         print("the basics of",keys,"is", topics[keys][2])
40
41 ### BEGIN TESTING
42
43 study_tool()
```